

Lab 10 Report

หัวข้อ	รายละเอียด
ชื่อ-สกุล / รหัสนักศึกษา	ปิยชญานิษฐ์ โทณะพันธ์
Section	2
โปรเจกต์	LAB10_WebClient — Product Management REST API

1. Reactive Programming vs Blocking I/O

Blocking I/O คือรูปแบบการเขียนโปรแกรมแบบดั้งเดิมที่เมื่อ thread เรียกใช้งาน operation ที่ต้องรอผล (เช่น query database, เรียก API ภายนอก, อ่านไฟล์) thread นั้นจะ หยุดรอ (block) จนกว่าจะได้ผลลัพธ์กลับมา ระหว่างที่รอ thread จะไม่สามารถไปทำงานอื่นได้เลย ทำให้ในระบบที่มี request จำนวนมากพร้อมกัน (high concurrency) ต้องใช้ thread จำนวนมากตามไปด้วย (โดยทั่วไปคือ 1 thread ต่อ 1 request) ซึ่งแต่ละ thread ก็กิน memory และ context-switching overhead

Reactive Programming คือรูปแบบการเขียนโปรแกรมแบบ asynchronous และ non-blocking ที่ยึดหลัก Publisher-Subscriber: แทนที่ thread จะรอผลลัพธ์ตรงๆ โปรแกรมจะ "สมัคร" (subscribe) เพื่อรับข้อมูลเมื่อมันพร้อม ผ่าน callback/operator chain แทน เมื่อ operation ที่ใช้เวลานาน (เช่น เรียก WebClient ไป service อื่น) ยังไม่เสร็จ thread จะถูกปล่อยกลับไปทำงานอื่นทันที (ไม่ block) แล้วเมื่อผลลัพธ์มาถึงค่อย "แจ้ง" กลับมาผ่าน event (onNext / onError / onComplete) ทำให้ระบบรองรับ concurrency สูงได้ด้วย thread pool ขนาดเล็ก (Spring WebFlux รันบน Reactor Netty แทน Tomcat แบบเดิม)

ตารางเปรียบเทียบ

ประเด็น	Blocking (แบบเดิม)	Reactive (WebFlux)
Thread model	รอ (block) จนกว่าจะได้ผลลัพธ์	ไม่รอ — thread กลับไปทำงานอื่นทันที
Scalability	1 thread ต่อ 1 request	Thread pool เล็ก รองรับหลาย request พร้อมกัน
Return type	String / Product (ค่าตรงๆ)	Mono<String> / Mono<Product>
เริ่ม execute เมื่อไร	ทันทีที่เรียก method	เมื่อเรียก .subscribe() เท่านั้น
Underlying server	Tomcat (Servlet, thread-per-request)	Reactor Netty (event-loop)
เหมาะกับ	CRUD ทั่วไป, งาน CPU-bound	High-concurrency I/O, streaming, microservices

กฎสำคัญของ **Reactive Stream**: Publisher (เช่น Mono/Flux) เป็นเพียง "blueprint" ของการทำงาน —

จะไม่มีอะไรเกิดขึ้นจริงจนกว่าจะมีการเรียก `.subscribe()` (หรือ Spring WebFlux เรียกแทนให้อัตโนมัติเมื่อ return จาก `@RestController`)

Publisher ส่ง signal ได้ 3 แบบ คือ `onNext(T)` ส่งข้อมูลได้ 0 ถึง N ครั้ง, `onError(e)` เกิดข้อผิดพลาดแล้ว stream จบทันที และ

`onComplete()` จบ stream แบบปกติ

2. Mono<T> vs Flux<T> — ใช้กรณีไหน และทำไม

Mono<T> คือ Publisher ที่ปล่อยค่าออกมา 0 หรือ 1 ค่าเท่านั้น แล้วจบ (onComplete) เหมาะกับ operation

ที่ตามธรรมชาติแล้วคืนผลลัพธ์เดียว เช่น หา record จาก id เดียว (findById), บันทึกข้อมูล 1 รายการ (save), หรือลบข้อมูล (delete ที่ไม่คืนค่า จะใช้ Mono<Void>) ในโปรเจกต์นี้ใช้ Mono กับ getById, save, delete, getDiscountedPrice

Flux<T> คือ Publisher ที่ปล่อยค่าออกมาได้ **0** ถึง **N** ค่า (stream ของหลาย element) เหมาะกับ operation ที่คืนหลายรายการ เช่น การดึงข้อมูลทั้งหมด (`findAll()`), ค้นหาแบบมีเงื่อนไขที่อาจได้หลายผล (`findByCategory()`) หรือแม้แต่ event stream ที่ยิงข้อมูลเข้ามาเรื่อยๆ ในโปรเจกต์นี้ใช้ Flux กับ `getAll` และ `getByCategory`

Rule of Thumb ในการเลือกใช้

จำนวนผลลัพธ์ที่คืน	ใช้ Type ไหน	ตัวอย่างใน Lab นี้
1 ผลลัพธ์ (หรือไม่มี)	<code>Mono<T></code>	<code>findById(id) -> Mono<Product></code>
หลายผลลัพธ์ (0..N)	<code>Flux<T></code>	<code>findAll() -> Flux<Product></code>
ไม่มีผลลัพธ์ (side-effect เช่น ลบ)	<code>Mono<Void></code>	<code>deleteById(id) -> Mono<Void></code>

เหตุผลที่ต้องแยก **type** สองแบบนี้แทนที่จะใช้ Flux ทุกที่ (ถึงแม้ Flux จะรองรับ 1 ค่าได้เช่นกัน) คือ Mono สื่อความหมาย (semantics) ให้ผู้ใช้ได้รู้ทันทีว่า "คาดหวังผลลัพธ์เดียว" และ Mono มี operator เฉพาะที่เหมาะสมกับ single-value เช่น `switchIfEmpty()` ที่ fallback เป็น Mono อีกตัวเมื่อว่าง — เช่นใน `getById()` ที่ใช้ `switchIfEmpty(Mono.error(...))` เพื่อโยน exception เมื่อหา product ไม่เจอ ส่วน Flux มี operator ที่เหมาะกับหลายค่า เช่น `filter()`, `take()`, `collectList()` ที่ใช้กรอง/จำกัด/รวม element

Operators สำคัญที่ใช้ในโปรเจกต์

Operator	ทำอะไร	ใช้ตรงไหนในโปรเจกต์
map	แปลงค่า T -> R แบบ synchronous	<code>getDiscountedPrice(): .map(Product::getDiscountedPrice)</code>
flatMap	แปลง T -> Publisher<R> แบบ asynchronous	ใช้เมื่อต้อง chain เรียก Mono/Flux ต่อกัน
filter	กรอง element ที่ผ่านเงื่อนไข	<code>findByCategory(): filter</code> ตาม category
switchIfEmpty	fallback เป็น Publisher อื่นถ้าว่าง	<code>getById(): throw error</code> ถ้าไม่พบสินค้า
defaultIfEmpty	fallback เป็นค่าคงที่ถ้าว่าง	WebClient demo: <code>.defaultIfEmpty("NOT FOUND")</code>
onErrorReturn	fallback ค่าเมื่อเกิด error	WebClient demo: <code>.onErrorReturn("NOT FOUND")</code>
subscribe	เริ่ม execute stream จริง (non-blocking)	ProductWebClient demo methods

3. WebClient — Method Chain แบบ Reactive

WebClient คือ HTTP client แบบ reactive ของ Spring ที่แทนที่ RestTemplate (แบบ blocking) เดิม จุดเด่นคือทุก method คืนค่าเป็น Mono/Flux ทำให้เรียก API ภายนอกได้โดยไม่ block thread ระหว่างรอ response โครงสร้าง method chain หลักมี 4 ขั้นตอนดังนี้

```
Mono<Product> result = client.get()
    .uri("/products/{id}", id)
    .retrieve()
    .bodyToMono(Product.class);
```

ตัวอย่างจาก `ProductWebClient.getProductById()`

ลำดับ	Method	หน้าที่
1	<code>.get()</code> / <code>.post()</code> / <code>.delete()</code>	กำหนด HTTP method ที่จะใช้เรียก (GET, POST, PUT, DELETE ฯลฯ) เป็นจุดเริ่มต้นของ request builder
2	<code>.uri("/products/{id}", id)</code>	กำหนด path ของ endpoint ปลายทาง รองรับ URI template (placeholder {id}) ที่จะถูกแทนที่ด้วย argument ที่ส่งเข้าไปตามลำดับ
3	<code>.retrieve()</code>	เริ่มกระบวนการรับ response กลับมา เป็นจุดที่บอกว่า "สนใจแค่ body" (ถ้าต้องการ header/status code ด้วยต้องใช้ <code>.exchangeToMono()</code> แทน) ยังไม่ execute request จริงจนกว่าจะมีการ subscribe
4	<code>.bodyToMono(T.class)</code> / <code>.bodyToFlux(T.class)</code>	แปลง response body (เช่น JSON) เป็น Mono<T> (ถ้าคาดหวังผลลัพธ์เดียว) หรือ Flux<T> (ถ้าคาดหวังหลายรายการ เช่น JSON array) โดยใช้ Jackson deserialize ให้อัตโนมัติ

สำคัญ: ทุก method ข้างต้น (get, uri, retrieve, bodyToMono/Flux) เป็นเพียงการ "ประกอบ" คำสั่ง (builder pattern) เท่านั้น — request จริงจะยังไม่ถูกยิงออกไปจนกว่าจะมีการ subscribe (เช่น ตอน Controller return Mono/Flux ให้ Spring WebFlux subscribe ให้อัตโนมัติ หรือเรียก `.subscribe()` เองตรงๆ) ทำให้สามารถ chain operator อื่น ต่อได้ทันทีก่อนถึงจุดที่ execute จริง เช่นตัวอย่างจาก `ProductWebClient` ที่ chain `.map()` -> `.defaultIfEmpty()` -> `.onErrorReturn()` -> `.subscribe()` ในคำสั่งเดียว โดย thread ที่เรียกจะไม่ค้างรอผลลัพธ์เลย

```
// call the endpoint, then chain operators immediately (non-blocking)
client.get()
    .uri("/products/{id}", id)
    .retrieve()
    .bodyToMono(Product.class)
    .map(p -> p.getName().toUpperCase()) // sync transform
    .defaultIfEmpty("NOT FOUND")         // fallback if Mono is empty
    .onErrorReturn("NOT FOUND")           // fallback if error (404/500)
    .subscribe(name -> System.out.println("Product name: " + name));
// returns immediately -- thread is free to do other work right away
```

ตัวอย่างจาก `ProductWebClient.printUpperCaseName()` — สาธิตการ chain operator แบบครบวงจร

4. Code: Repository / Service / Controller

4.1 ProductRepository — In-memory Reactive Repository

ทำหน้าที่เก็บและดึงข้อมูล Product จาก ConcurrentHashMap ในหน่วยความจำ (จำลองแทน database) โดยทุก method คืนค่าเป็น Mono/Flux ตามจำนวนผลลัพธ์ที่คาดหวัง

```
public class ProductRepository {

    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // find 1 item -> Mono (0 or 1 value)
    public Mono<Product> findById(String id) {
        Product product = store.get(id);
        // if not found (null), return Mono.empty() instead of null,
        // so callers can handle the "empty" case with operators
        // such as switchIfEmpty()
        return product == null ? Mono.empty() : Mono.just(product);
    }

    // find everything -> Flux (0..N values)
    public Flux<Product> findAll() {
        return Flux.fromIterable(store.values());
    }

    // save a Product -> Mono<Product> (returns what was saved)
    public Mono<Product> save(Product product) {
        store.put(product.getId(), product);
        return Mono.just(product);
    }

    // delete -> Mono<Void> (no data returned, just a completion signal)
    public Mono<Void> deleteById(String id) {
        store.remove(id);
        return Mono.empty();
    }

    // filter by category -> Flux built on top of findAll() + filter()
    public Flux<Product> findByCategory(String category) {
        return findAll()
            .filter(p -> p.getCategory() != null
                && p.getCategory().equalsIgnoreCase(category));
    }
}
```

4.2 ProductService — Business Logic Layer

ชั้น Service ทำหน้าที่เพิ่ม business logic ก่อนส่งต่อไปยัง Repository เช่น การจัดการ error เมื่อไม่พบข้อมูล และการ generate id อัตโนมัติ โดยใช้ Constructor Injection รับ ProductRepository เข้ามา (DIP — SOLID)

```
@Service
public class ProductService {

    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository; // Constructor Injection (DIP)
    }
}
```

```

// find 1 item; throw an error instead of silently returning empty
public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(Mono.error(
            new RuntimeException("Product not found: " + id)));
}

public Flux<Product> getAll() {
    return repository.findAll();
}

// if the client didn't send an id, generate a new UUID before saving
public Mono<Product> save(Product product) {
    if (product.getId() == null || product.getId().isBlank()) {
        product.setId(java.util.UUID.randomUUID().toString());
    }
    return repository.save(product);
}

public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// fetch the Product first (via getById, which already handles errors)
// then map to just the discounted price
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(Product::getDiscountedPrice);
}
}

```

4.3 ProductController — Reactive REST Controller

Controller เป็นชั้นบนสุดที่รับ HTTP request แล้วส่งต่อไปยัง Service เพียงอย่างเดียว (thin controller) โดยคืน Mono/Flux ตรงๆ ให้ Spring WebFlux เป็นผู้ subscribe และ serialize เป็น JSON ให้อัตโนมัติแบบ non-blocking

```
@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service; // Constructor Injection (DIP)
    }

    // GET /products -> return everything as Flux<Product>
    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    // GET /products/{id} -> return 1 item as Mono<Product>
    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }

    // POST /products -> read Product from the request body, then save it
    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }

    // DELETE /products/{id} -> delete, return Mono<Void> (no body)
    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) {
        return service.delete(id);
    }

    // GET /products/category/{category} -> filter by category
    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) {
        return service.getByCategory(category);
    }

    // GET /products/{id}/price -> discounted price
    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) {
        return service.getDiscountedPrice(id);
    }
}
```

สรุป **data flow**: Controller (รับ HTTP, ไม่มี logic) -> Service (business logic, error handling) -> Repository (data access) โดยทุกชั้นสื่อสารกันด้วย Mono/Flux ตลอดทั้ง chain ไม่มีจุดใดเลยที่เรียก `.block()` ทำให้ thread ของ Netty event-loop ไม่ถูกกักไว้รอผลลัพธ์ ณ จุดใดจุดหนึ่ง — นี่คือหัวใจของการทำ REST API แบบ non-blocking ด้วย Spring WebFlux

5. ชื่อและรหัสนักศึกษาใน response (Product name ที่ seed ไว้)

โจทย์กำหนดให้ผลลัพธ์ (response) ที่ได้จาก endpoint ต้องมีชื่อและรหัสนักศึกษาปรากฏอยู่ — วิธีที่ง่ายและตรงจุด ที่สุดคือใส่ไว้ใน **name** ของ Product ที่ seed ไว้ตอนสร้าง **ProductRepository** (ใน constructor) เพราะทุกครั้งที่เราเรียก **GET /products** หรือ **GET /products/1** ชื่อสินค้านี้จะถูกคืนกลับมาใน JSON response โดยอัตโนมัติ

ในไฟล์ template ที่ให้มามี placeholder เตรียมไว้แล้วที่ product แรก (id = "1") ในรูปแบบ **"67XXXXXXXX-X SEC 1"** — นักศึกษาต้องแก้ค่านี้เป็นชื่อ-รหัสนักศึกษาจริงของตัวเอง ก่อนส่งงาน ตัวอย่างเช่น เปลี่ยนจาก

```
// — ก่อนแก้ (ค่าเดิมใน template) —————
store.put("1", new Product("1", "iPhone 15 Pro (67XXXXXXXX-X SEC 1)",
    "Electronics", "Apple", 50, 39900.0, "MEMBER"));
```

src/main/java/com/example/lab10/repository/ProductRepository.java — constructor

```
// — หลังแก้ (ตัวอย่าง) —————
store.put("1", new Product("1", "iPhone 15 Pro (65010000-1 SEC 1)",
    "Electronics", "Apple", 50, 39900.0, "MEMBER"));
```

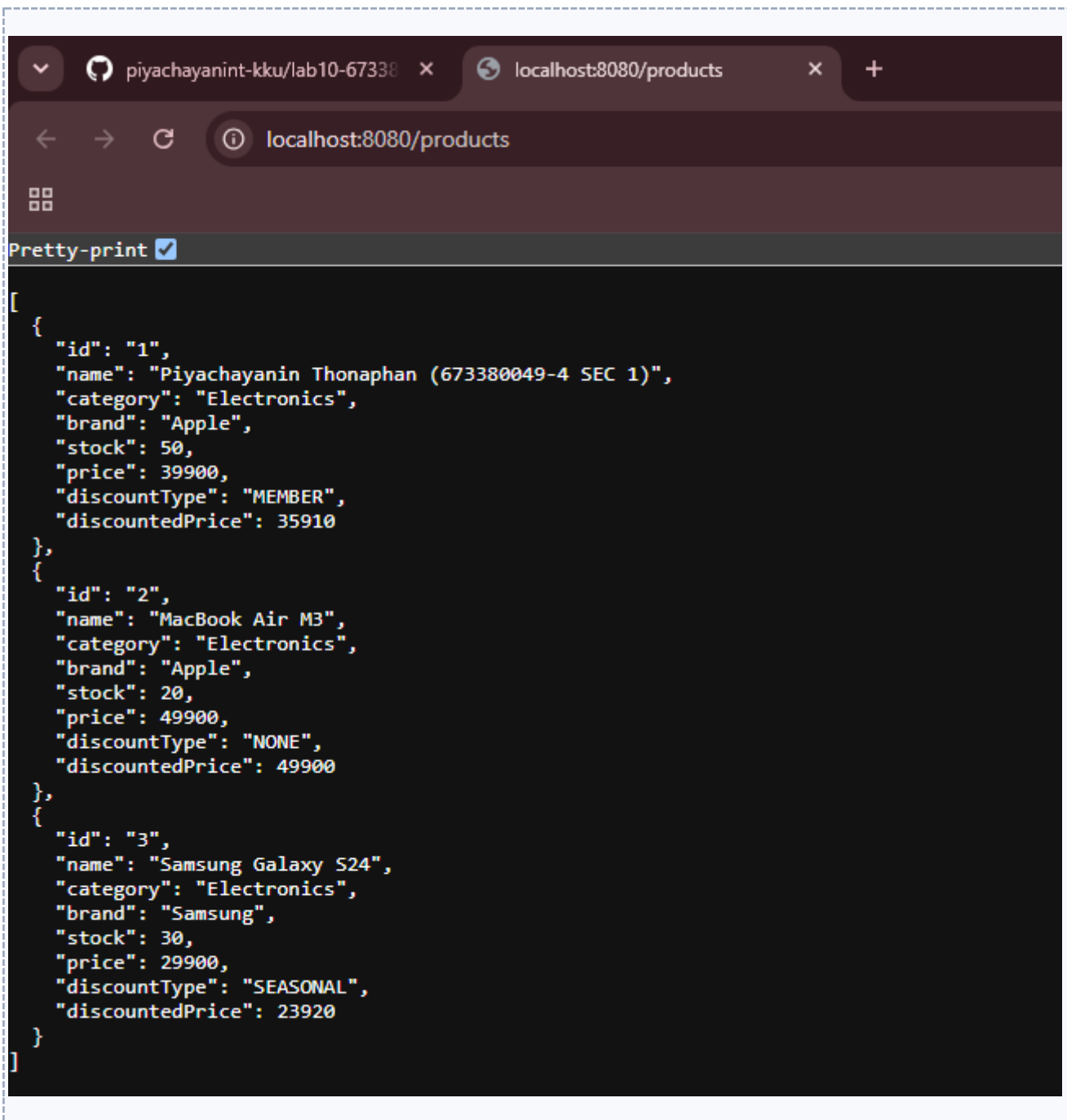
แทนที่ 67XXXXXXXX-X ด้วยรหัสนักศึกษาจริง และ SEC 1 ด้วย section จริงของนักศึกษา

เมื่อแก้แล้ว ให้ทดสอบเรียก **GET /products/1** หรือ **GET /products** อีกครั้ง แล้วตรวจสอบว่าค่า **name** ใน JSON response แสดงชื่อ-รหัสนักศึกษาที่ถูกต้องแล้ว ก่อนจะกดหน้าจอส่งในหัวข้อถัดไป

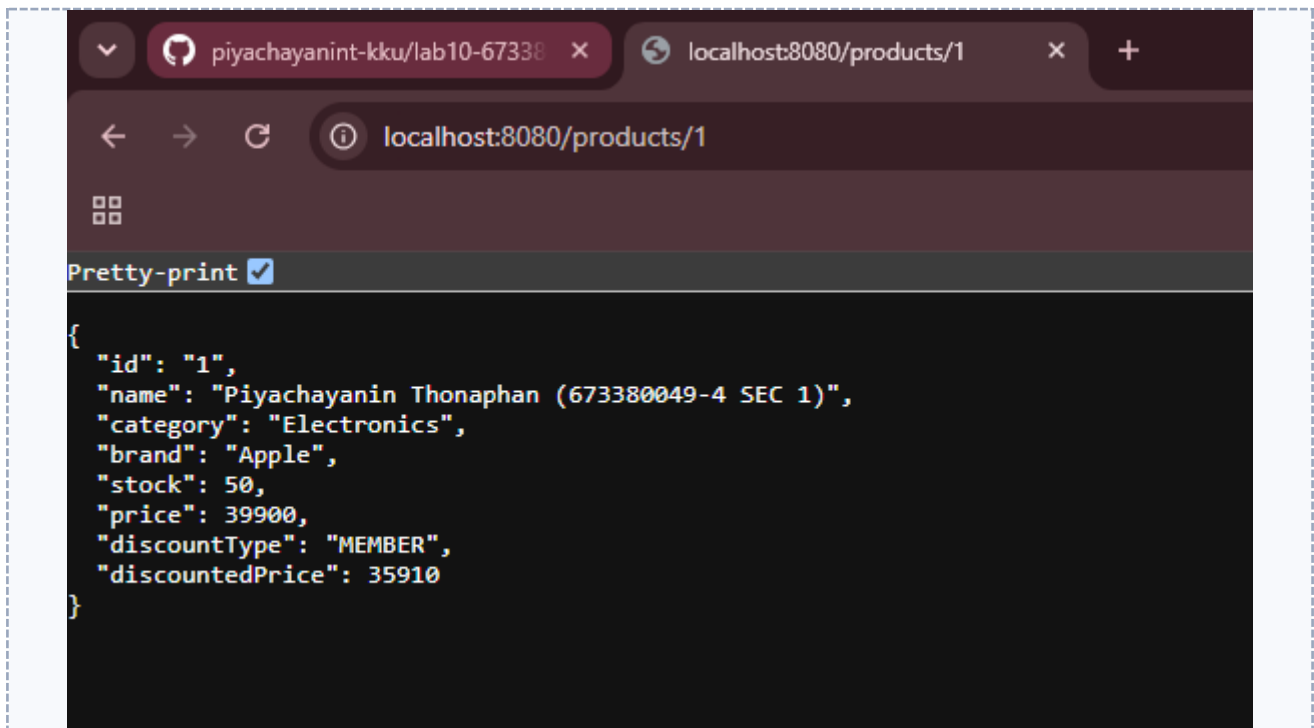
6. Screenshot ทดสอบทุก Endpoint

หมายเหตุ : ในภาพ Screenshot ใส่เลข Section ผิด เลขจริงคือ Sec 2

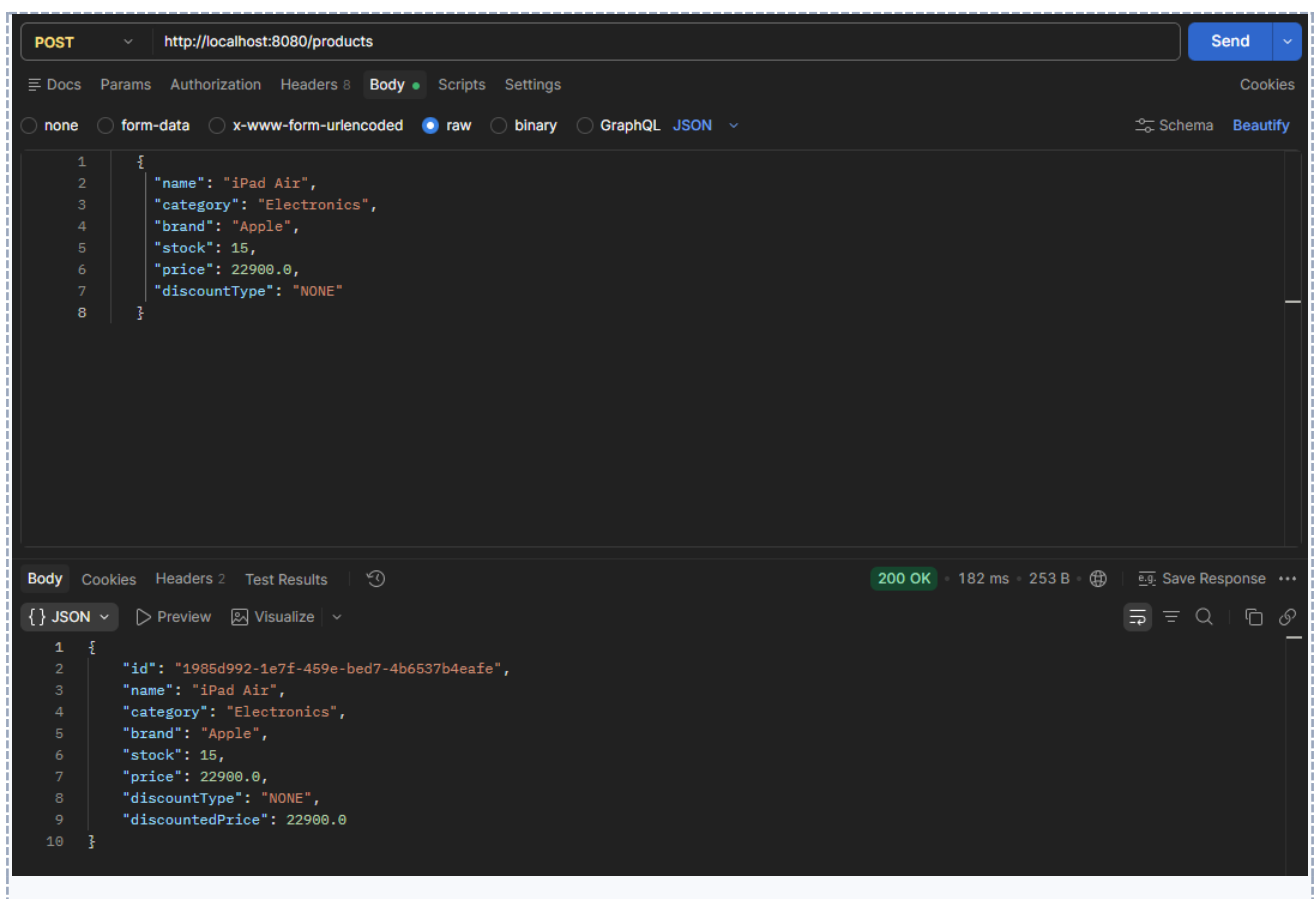
6.1 GET /products



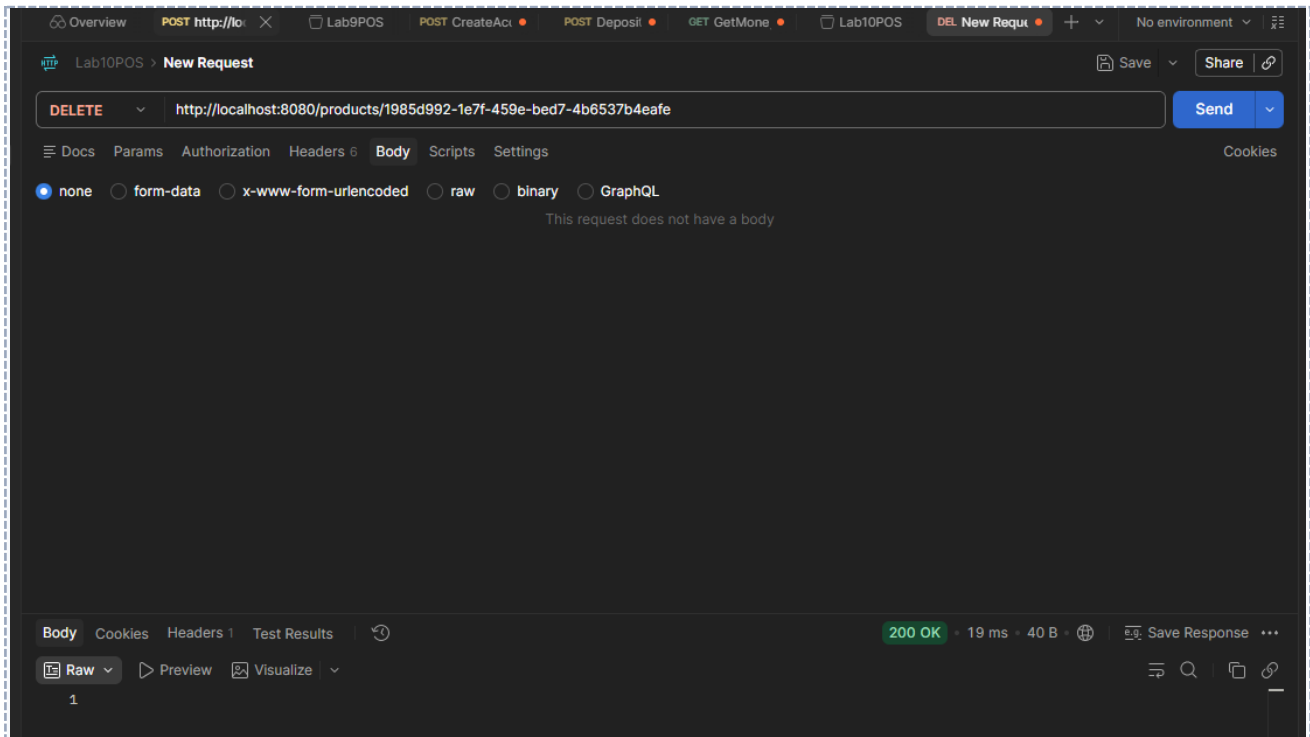
6.2 GET /products/1



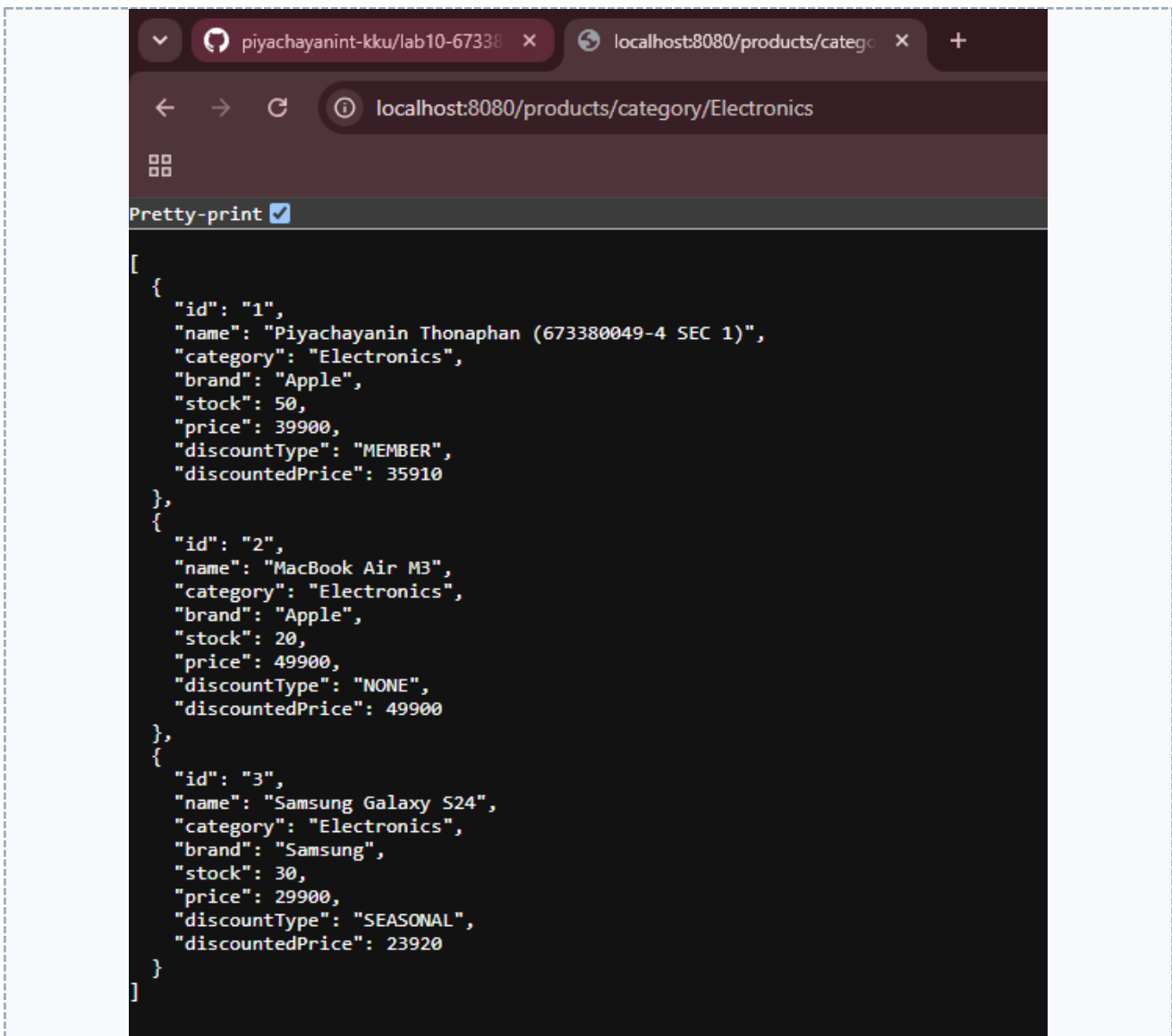
6.3 POST /products



6.4 DELETE /products/1



6.5 GET /products/category/Electronics



6.6 GET /products/1/price

